SENTINELAI

A Multi-Agent Urban Digital Twin for Predictive Disaster Intelligence and Autonomous Emergency Response

A CAPSTONE PROJECT REPORT

Submitted in partial fulfilment of the requirements for the Course of

CSA1709 – ARTIFICIAL INTELLIGENCE

to the award of the degree of BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING

Submitted by

Santhosh Kumar B [192511053]

G. Madhan Kumar [192472328] |

Avishek N [192511009]

Under the Supervision of

Dr. Rajaram P

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "SentinelAI: A Multi-Agent Urban Digital Twin for Predictive Disaster Intelligence and Autonomous Emergency Response" has been carried out by B. Santhosh Kumar [192511053], G. Madhan Kumar [192472328] and Avishek N [192511009] under the supervision of Dr. Rajaram P and is submitted in partial fulfilment of the requirements for the current semester of the B.E. Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY
DR. KRISHNA KUMAR KATHIRESAN	Dr Rajaram P
Professor	Professor
CSE/Generative AI	CSE/Data Vista,
SIMATS Engineering,	SIMATS Engineering,
SIMATS, Chennai – 602105.	SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

DECLARATION BY THE CANDIDATE

We, Santhosh Kumar B [192511053], G. Madhan Kumar [192472328], and Avishek N [192511009] of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "SentinelAI: A Multi-Agent Urban Digital Twin for Predictive Disaster Intelligence and Autonomous Emergency Response" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Chennai
Date: 5/09/2026

Signature of the Candidate

Santhosh Kumar B [192511053]

G. Madhan Kumar [192472328]

Avishek N [192511009]

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
B. Santhosh Kumar 192511053	Project architecture, end-to-end integration, core intelligence workflow, frontend/backend coordination, Digital Twin integration, security/RBAC hardening, deployment verification and final system validation.	Designed the end-to-end architecture; integrated ESP32 → Wi-Fi → Django REST → PostgreSQL/PostGIS → risk engine → Digital Twin/dashboard flow; coordinated incident, officer, routing and citizen workflows; implemented/verified core integration and security behaviour.	Performed end-to-end functional verification, role/RBAC checks, sensor-ingestion checks, incident workflow testing, Digital Twin/map verification, build/runtime checks and debugging of integration failures.	Prepared the individual contribution report, architecture explanation, testing tables, risk/security discussion, reflection and final documentation.	60%
G. Madhan Kumar 192472328	Supporting implementation and module development within the team project.	Contributed to assigned software modules and feature implementation under the common project architecture.	Supported module-level testing and validation.	Contributed technical notes and project documentation.	20%
Avishek N 192511009	Supporting implementation, validation and project activities within the team.	Contributed to assigned modules and integration activities.	Supported functional testing and demonstration preparation.	Contributed technical documentation and review.	20%
Total					100%

ABSTRACT

Urban disaster response requires rapid interpretation of changing environmental conditions, spatial impact and responder availability. Conventional workflows can become fragmented when sensor observations, incident reports, maps and response teams are managed through separate processes. This project develops SentinelAI, a multi-agent urban digital twin prototype that connects physical sensing, data ingestion, risk evaluation, geospatial visualization and emergency-response coordination in one workflow. The implemented prototype uses an ESP32-based sensor layer for rainfall, water level, flame, gas and temperature/humidity demonstrations. Sensor observations are transmitted through Wi-Fi to a Django REST backend and stored in a PostgreSQL/PostGIS data layer. An intelligence layer evaluates configurable thresholds and risk indicators, while the Digital Twin presents zone status and supports operational decision-making. The response layer creates incidents, recommends available officers, supports escalation, and uses route calculation for response planning. Citizen reporting and public-token tracking extend the platform beyond internal command-centre monitoring. The system was validated through prototype scenarios covering threshold-triggered incidents, sensor disconnection, officer availability, blocked-road routing, citizen reporting and role-based access. The results confirm that the integrated architecture can convert sensor events into coordinated software actions without relying on manually entering each intermediate state. The project remains a prototype and does not claim city-scale field accuracy or a validated production machine-learning accuracy score. The principal contribution of this individual report is the architecture and integration work that connects the sensing, intelligence, Digital Twin and response layers into a coherent demonstration-ready system.

Keywords: Urban Digital Twin, Disaster Intelligence, Multi-Agent Systems, IoT, Geospatial Risk, Emergency Response

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	—
A*	A-star path-finding algorithm	—
API	Application Programming Interface	—
Django REST / DRF	Django REST Framework	—
GIS	Geographic Information System	—
IoT	Internet of Things	—
JWT	JSON Web Token	—
ML	Machine Learning	—
MQ-2	Metal Oxide Gas Sensor	—
PostGIS	Spatial extension for PostgreSQL	—
RBAC	Role-Based Access Control	—
REST	Representational State Transfer	—
ESP32	Wi-Fi/Bluetooth-enabled microcontroller	—
WebSocket	Persistent bidirectional communication channel	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Urban environments contain interacting physical, social and digital systems. Weather changes, water accumulation, fire hazards, gas leakage, traffic disruption and responder availability can evolve simultaneously. A disaster-management platform therefore needs more than a static map or a simple alarm: it needs a mechanism for observing conditions, interpreting risk, representing affected areas and coordinating the next operational action.

Digital-twin research positions a digital representation as a way to connect physical-world observations with computational models and decision support. Batty discusses digital twins in the context of urban analytics, while Li, Yu and Shao describe digital-twin cities as an important direction for smart-city development [1], [3]. Fan et al. specifically propose a Disaster City Digital Twin vision that combines multimodal data, artificial intelligence and human decision-making for disaster management [2]. These research directions motivate the architecture adopted in SentinelAI.

The implemented project combines a physical prototype with a web-based software platform. The physical layer uses an ESP32 and environmental sensors. The software layer receives observations, stores them, evaluates risk, updates a geospatial Digital Twin and supports incident and responder workflows. The resulting architecture follows a continuous sense–predict–simulate–decide–respond loop rather than treating detection and response as separate applications.

1.2 Need for the Project

- Disaster information is frequently distributed across sensors, reports, maps, weather observations and responder records.
- Manual monitoring can delay the transition from an observed hazard to an actionable incident.
- Static maps cannot represent changing risk conditions, blocked roads or evolving response requirements.
- Simultaneous incidents create a prioritization problem because responder availability, distance, skill and workload interact.

- Citizens need a controlled mechanism to report incidents and track progress without receiving internal operational information.

The engineering need is therefore an integrated intelligence layer that can convert heterogeneous observations into risk information, spatial status, prioritized incidents, responder recommendations, route guidance and citizen-safe notifications. This is consistent with the project design record, which identifies reactive monitoring, disconnected data sources, manual incident creation and static maps as major gaps [project record].

1.3 Problem Statement

Design and implement a prototype urban disaster-intelligence system that continuously receives environmental observations, evaluates hazard risk, represents the affected zone through a Digital Twin and coordinates emergency-response actions under realistic constraints of limited sensors, uncertain readings, network dependence, privacy requirements and prototype-scale resources.

The problem is complex because the system must simultaneously satisfy low-latency data handling, reliable state transitions, spatial reasoning, role-specific access, responder allocation, route calculation and safe communication. Increasing automation must not remove human authority from high-impact emergency decisions.

1.4 Project Aim

To design and develop SentinelAI, an AI-assisted urban disaster management platform integrating IoT sensing, predictive risk analysis, geospatial Digital Twin visualization and coordinated emergency response to improve situational awareness and reduce avoidable response delays in a prototype environment.

1.5 Project Objectives

1. Design an end-to-end architecture connecting physical sensors, data ingestion, intelligence, geospatial representation and response coordination.
2. Collect and process environmental readings from the ESP32 prototype.
3. Detect abnormal conditions and generate configurable risk states and incidents.
4. Represent monitored zones, assets and risk conditions through a GIS-enabled Digital Twin.
5. Support automatic responder recommendation using availability, zone, distance, skill and workload factors.

6. Provide route calculation for emergency response and evacuation planning.
7. Provide citizen incident reporting with controlled evidence and public-token tracking.
8. Validate the system through functional, security and end-to-end prototype tests.
9. Document the engineering trade-offs, risks, limitations and individual contribution.

1.6 Scope

Included scope: prototype sensing for rain, water level, flame, gas and temperature/humidity; REST-based sensor ingestion; persistent data storage; configurable risk evaluation; Digital Twin/map visualization; incident lifecycle; responder recommendation; route planning; citizen reporting; role-based access; notifications; analytics and audit-oriented records.

Excluded scope: certified city-scale deployment, guaranteed disaster prediction accuracy, autonomous physical rescue, official emergency-service dispatch, unrestricted live CCTV, operational satellite feeds and a fully validated city-wide machine-learning model. The project records explicitly treat advanced satellite, CCTV, drone and 3D Digital Twin capabilities as future extensions.

Assumptions and boundaries include prototype sensor coverage, controlled test scenarios, available network connectivity during normal operation, configured thresholds, prototype zone labels and human verification for high-impact decisions.

1.7 Expected Engineering Outcome

The intended outcome is a working prototype that demonstrates a complete data-to-decision chain: sensor event → authenticated transmission → backend validation and storage → risk evaluation → Digital Twin update → incident generation → responder recommendation → route calculation → notification and audit state. The outcome is therefore both a software platform and a physical-to-digital integration prototype.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review was conducted using peer-reviewed journal and conference records, publisher pages, institutional repositories and ResearchGate/Google Scholar-indexed records. Search themes included urban digital twins, disaster-management information systems, multi-agent coordination, IoT sensing, crowdsourced disaster information, geospatial risk mapping and machine-learning-based flood prediction. The review emphasized papers with identifiable publication details and DOI information.

The project sources were compared against the implemented system rather than copied as text. The review was used to identify the engineering gap and to justify the combination of sensing, spatial representation, prediction and response coordination.

2.2 Review of Existing Work

Batty [1] discusses the role of digital twins within the urban-computing landscape and establishes the conceptual relevance of continuously connected digital representations. Fan et al. [2] extend the idea to disaster management by proposing a Disaster City Digital Twin that integrates multimodal data and AI to support situational awareness, decision-making and coordination. These works support the use of a Digital Twin as a decision-support layer rather than merely a visualization.

Li, Yu and Shao [3] examine smart-city development based on digital twins and identify applications enabled by combining digital representations with data and intelligent services. Their work reinforces the value of connecting spatial models to operational information.

Atzori, Iera and Morabito [4] and Al-Fuqaha et al. [5] provide broad foundations for IoT systems. They emphasize the integration of sensing, communications, protocols and distributed intelligence. SentinelAI applies these principles at prototype scale by using an ESP32 sensing node and a network-facing API rather than treating sensor values as isolated local measurements.

Ramchurn et al. [6] describe HAC-ER, a disaster-response system based on human-agent collectives. The work is particularly relevant to SentinelAI because it demonstrates how software

agents and humans can cooperate in disaster response rather than assuming that automation should replace human command.

Kankanamge et al. [7] demonstrate that social-media analysis can contribute citizen-level disaster situation awareness and spatial understanding. SentinelAI incorporates a related principle through controlled citizen reporting, photo evidence and public-token tracking, while keeping operational information protected.

Natsvlishvili, Jorjiashvili and Kochoradze [8] present a PostGIS-based approach to natural-disaster risk mapping, showing the value of spatial SQL and open-source geospatial databases for risk analysis. This supports the use of PostgreSQL/PostGIS in SentinelAI.

Yan et al. [9] demonstrate rapid urban flood prediction by coupling machine learning and numerical simulation. The study reinforces the value of predictive flood analysis, but also highlights the need for suitable data and modelling assumptions. SentinelAI therefore avoids claiming a field-validated prediction accuracy and instead uses a configurable prototype risk engine while leaving richer spatiotemporal learning for future work.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to SentinelAI
Static disaster dashboard	Simple visualization; easy to deploy	Reactive; weak integration with sensing and response	Motivates live Digital Twin and event-driven updates
Urban Digital Twin research [1]–[3]	Spatial representation; simulation and decision support	Often requires large data and city-scale infrastructure	Provides conceptual basis for Digital Twin layer
IoT-only monitoring [4], [5]	Continuous sensing; low-cost edge devices	Does not inherently coordinate response	Provides sensing and communication foundation
Human-agent disaster response [6]	Combines automation with human decision-making	Complex coordination and deployment requirements	Motivates agent-based incident and responder coordination
Crowdsourced disaster analytics [7]	Adds citizen-level observations	Quality, privacy and geolocation uncertainty	Motivates controlled citizen reporting and evidence
PostGIS risk mapping [8]	Strong spatial query and risk-map capability	Requires structured spatial data	Supports spatial zone and route processing
ML flood prediction [9]	Predictive capability for flood conditions	Needs training data and validation	Supports future predictive analytics; current prototype remains transparent and configurable

2.4 Research / Engineering Gap

The reviewed literature provides strong solutions for individual dimensions of disaster intelligence, but a college-scale prototype still needs a practical integration path that can be demonstrated physically and software-wise. The gap addressed by SentinelAI is the integration of low-cost IoT sensing, configurable risk analysis, a geospatial Digital Twin, incident state management, responder allocation, routing and citizen feedback in a single end-to-end workflow.

A second gap is traceability. A system may detect risk correctly but still fail operationally if the event is not converted into an incident, assigned to an available responder, routed safely and communicated to the appropriate user. SentinelAI therefore treats the transition between modules as an engineering requirement.

2.5 Proposed Contribution

The proposed contribution is an integrated prototype architecture in which physical sensor changes can propagate through the backend and intelligence layer to produce a visible Digital Twin state and operational response. The project also introduces role-aware access, incident lifecycle management, zone-wise responder recommendation, route calculation and citizen-token tracking as connected rather than isolated features.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder	Requirement
Command centre / administrator	Unified view of zones, incidents, sensor status, resources, alerts and analytics.
Disaster-management officer	Rapid risk interpretation, incident visibility and responder coordination.
Emergency responder	Assigned incident, response timer, route and relevant operational information.
Citizen	Safe public warnings, incident reporting, photo evidence and token-based status tracking.
System administrator	Configurable thresholds, role management, audit visibility and system health.
Project evaluator	Reproducible prototype workflow with visible physical and software evidence.

Requirement	Type	Measurable Target
Sensor data ingestion	Functional	Valid sensor readings accepted through the defined API and persisted.
Risk evaluation	Functional	Threshold/risk changes reflected in zone state and incident logic.
Digital Twin	Functional	Zone risk and operational state update from stored system data.
Responder allocation	Functional	Unavailable responders excluded; eligible responder recommended.
Routing	Functional	Blocked/unsafe road nodes excluded from the selected route graph.
RBAC	Non-Functional/Security	Unauthorized protected actions rejected; citizen view excludes internal data.
Reliability	Non-Functional	Failure states handled without crashing the main monitoring workflow.
Maintainability	Non-Functional	Modules separated by sensing, API, intelligence, geospatial and response responsibilities.

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE 29148	Requirements engineering	Functional and non-functional requirements expressed before implementation and verification.
ISO/IEC 27001 principles	Information security	Role-based access, least privilege, protected citizen evidence and controlled operational information.
ISO 22320	Emergency management	Incident coordination, information sharing, roles and response-oriented workflows.
OWASP secure development principles	Application security	Input validation, protected endpoints, secure file handling and avoidance of privilege escalation.
OGC-compatible geospatial concepts	Spatial data interoperability	Zone geometry and spatial operations represented through PostgreSQL/PostGIS.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Sensor communication	Wi-Fi based HTTP/REST ingestion	—	High
Database	PostgreSQL with spatial capability	—	High
Frontend	Role-aware web dashboard	—	High
Risk states	Normal / warning / critical as configured	state	High
Incident lifecycle	Detect → verify/prioritize → assign → accept → progress → resolve/close	state	High
Responder selection	Availability + distance + skill + workload + configured performance factors	score	High
Route planning	Avoid configured blocked/unsafe nodes	path	High
Citizen evidence	Controlled image upload + public token	record	Medium

Realtime events	Sensor/risk/incident/assignment updates	event	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: A simple standalone IoT dashboard would display sensor values and alerts but would not provide integrated spatial reasoning or responder coordination.

Alternative 2: A cloud-first AI platform could support larger data volumes and advanced models, but would introduce higher deployment complexity and would make the physical prototype harder to demonstrate within the project constraints.

Alternative 3: The selected modular web architecture combines an ESP32 sensing layer, Django REST backend, PostgreSQL/PostGIS, intelligence services, Digital Twin visualization and response workflows. It provides a practical balance between demonstrability, extensibility and engineering complexity.

Criterion	Weight	Standalone IoT Dashboard	Cloud-first AI Platform	Selected Modular Prototype
Performance	25%	Medium	High	High for prototype
Cost	20%	High	Low–Medium	High
Safety / Control	20%	Medium	Medium	High with RBAC and human verification
Sustainability	15%	Medium	Medium	High through modular/open-source stack
Reliability / Debuggability	20%	Medium	Medium	High for controlled prototype
Overall	100%	Moderate	Moderate–High	Best fit for capstone constraints

3.5 Selected Design & Detailed Design

The selected design uses a layered architecture. The sensing layer contains ESP32 and environmental sensors. The data layer authenticates, validates and stores observations. The intelligence layer evaluates risk and anomaly indicators. The geospatial layer maps zones, assets

and routes. The decision layer handles incident priority and responder recommendation. The communication layer serves officers, administrators and citizens. The analytics layer records trends and response performance.

The physical-to-digital interface is the most important integration boundary. The ESP32 reads sensor values and transmits them through Wi-Fi. The backend validates the payload, stores the reading and triggers downstream evaluation. The risk state is then reflected in the Digital Twin and incident workflow. This avoids manually entering intermediate dashboard states and makes the demonstration traceable from physical event to software response.

The architecture is modular so that the intelligence engine can be replaced or expanded without rewriting the sensor and response layers. Similarly, additional data sources such as satellite or authorized CCTV can be added later as new ingestion modules.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
REST-based sensor ingestion	MQTT-only prototype	Simple integration and debugging	Less suitable for very large device fleets	REST selected for prototype; MQTT retained as future scale option.
PostgreSQL/PostGIS	Non-spatial relational DB	Spatial queries and structured records	Requires spatial setup	Selected because zone/route operations are spatial.
Configurable risk rules + predictive extension	Black-box ML-only model	Transparent and testable with limited data	Lower predictive sophistication	Selected for honest prototype validation; richer ML reserved for future work.
Role-aware application	Single common dashboard	Simple UI	Weak privacy	Role-aware design selected to protect operational data.
A* route planning	Manual route selection	Repeatable and algorithmic	Depends on road graph quality	Selected for prototype evacuation/response routing.

3.7 Sustainability, Ethics, Safety & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Use reusable/open technologies and modular components.	ESP32, Python, Django, PostgreSQL/PostGIS, Leaflet and modular software architecture.	Reduced dependency on proprietary infrastructure and supports extension.
Ethics	Avoid replacing authorized emergency decisions with autonomous AI.	Human verification requirement and transparent prototype risk logic.	AI is positioned as decision support, not final command authority.
Health & Safety	Incorrect alerts or routes can create harm.	Threshold testing, route constraints, escalation and explicit limitations.	High-impact actions remain subject to human verification.
Security	Citizen images, locations and responder information are sensitive.	RBAC, authenticated APIs, least privilege, file validation and audit-oriented records.	Public views expose only citizen-safe information.

3.8 Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Sensor noise / false spike	Medium	Medium	Medium	Calibration, validation and anomaly handling	Low–Medium
Network interruption	Medium	High	High	Retry/buffering strategy and failure-state handling	Medium
False positive alert	Medium	High	High	Configurable thresholds + human verification	Medium

Incorrect route due to stale graph	Low–Medium	High	Medium–High	Road-status validation and route testing	Medium
Unauthorized data access	Low	High	High	RBAC, API permissions, least privilege	Low–Medium
Limited training data	High	Medium	High	Transparent baseline logic; do not claim unsupported ML accuracy	Medium
Prototype hardware failure	Medium	Medium	Medium	Physical inspection and fallback test plan	Low–Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development followed an incremental integration approach. The system was first decomposed into sensing, backend, intelligence, geospatial and response responsibilities. Interfaces were then connected through defined API contracts and persistent data models. Testing was performed from individual module behaviour toward the complete physical-to-digital workflow. This approach was particularly important because a successful frontend build alone does not prove that sensor ingestion, database persistence, risk evaluation, WebSocket events, role permissions and incident state transitions work together.

Resource / Component	Purpose
ESP32 DevKit	Microcontroller for prototype sensing and Wi-Fi transmission
Rain sensors	Rain/flood demonstration input
Water-level sensor	Water accumulation / flood demonstration input
Flame sensor	Fire detection demonstration input
MQ-2 gas sensor	Gas-risk demonstration input
DHT11/DHT22-class temperature-humidity sensor	Heat/weather demonstration input
Django + Django REST Framework	Backend API and business logic
PostgreSQL + PostGIS	Persistent and spatial data storage
React + TypeScript	Role-aware web interface
Leaflet / OpenStreetMap	Map and Digital Twin visualization
Python intelligence services	Risk evaluation, anomaly/prediction extensions
WebSocket / Django Channels	Live event propagation where enabled
A* path planning	Response and evacuation route calculation

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

The physical prototype was assembled around an ESP32 on a breadboard with zone-labelled sensors. The arrangement was designed to make the relationship between physical sensing and software response visible during demonstration. The image below is the actual project hardware evidence available in the project report and is not a stock or generated illustration.

The prototype includes Zone A Velachery for rain and water-level monitoring, Zone B Tambaram for rain monitoring, Zone C Guindy for flame/fire monitoring, Zone D Industrial Area for MQ-2 gas monitoring, and Zone E T. Nagar for temperature/humidity monitoring. These are prototype demonstration labels and are not claims that the named locations have been officially classified as disaster-risk zones.

Tool / Software / Equipment	Purpose	Stage Used
ESP32 + sensors	Physical data acquisition	Prototype integration
Django REST Framework	Sensor API, incidents, users and response services	Backend implementation
PostgreSQL/PostGIS	Persistent records and spatial data	Database implementation
React + TypeScript	Role-specific user interface	Frontend implementation
Leaflet + OpenStreetMap	Digital Twin / map visualization	Geospatial implementation
Python	Risk and data-processing logic	Intelligence layer
A* algorithm	Dynamic response/evacuation route planning	Decision layer
Git/GitHub	Version control and integration	Development management
Vercel / deployment tooling	Frontend deployment verification	Deployment stage

4.3 Test Plan & Verification

The test plan was defined around the system's observable transitions rather than only code compilation. Each test checks whether an input creates the expected state change while preserving security and data integrity.

Requirement	Test Method	Acceptance Criterion
Rain sensor event	Increase Zone A rain input	Zone risk increases according to configured logic.
Water-level threshold	Raise water-level condition	Flood-related incident is generated.
Flame trigger	Activate flame sensor	Fire-related incident is generated.
Gas threshold	Raise MQ-2 condition	Gas-risk incident is generated.
Sensor disconnect	Disconnect device / stop readings	Device becomes offline or stale according to health logic.

False sensor spike	Introduce abnormal single reading	Anomaly warning or validation state prevents blind escalation.
Officer unavailable	Mark preferred responder unavailable	Next eligible responder is recommended.
Response timeout	Allow response timer to expire	Escalation path is triggered.
Road blocked	Mark route node/segment unavailable	A* route avoids blocked path where graph permits.
Citizen report	Submit photo report	Report record and public token are generated.
Unauthorized access	Attempt protected operation using restricted role	Access denied / HTTP 403 at protected API boundary.
Incident resolution	Resolve an incident	State, audit information and performance/points record update.

4.4 Results

Test Case	Expected Result	Observed / Verified Result	Status
Hazard threshold exceeded	Alert / incident generated	Prototype record confirms threshold-triggered alert behaviour	PASS
Normal conditions	No unnecessary alert	Alert state clears/returns to normal when conditions normalize	PASS
High risk condition	Response recommendation issued	Coordination workflow produces response action/recommendation	PASS
Sensor-to-backend path	Reading accepted and persisted	ESP32 → Wi-Fi → backend → database path maintained in project implementation	PASS
Digital Twin update	Zone reflects risk state	Risk/zone state is connected to Digital Twin workflow	PASS
Officer allocation	Eligible officer selected	Zone, availability and workload logic used for recommendation	PASS
Citizen report	Token-based tracking	Citizen reporting workflow includes photo/token/status concepts	PASS

Security/RBAC	Unauthorized access blocked	Role restrictions and protected endpoints tested during project hardening	PASS

The project records do not provide a statistically validated city-scale accuracy percentage, so no unsupported accuracy value is reported here. The meaningful result for this capstone is the successful integration of the end-to-end prototype workflow and the repeatable functional verification of its major transitions.

4.5 Data Analysis & Engineering Judgment

The most important engineering observation was that failures at module boundaries are more consequential than failures inside an isolated component. For example, a sensor reading is useful only if it is transmitted with the expected schema, stored correctly, interpreted using the current threshold configuration and reflected in the correct zone state. Likewise, a risk state has limited value if it does not become an incident or if the incident cannot be assigned to an available responder.

The testing therefore emphasized state propagation. A successful test was treated as evidence that the next layer received and interpreted the previous layer's output correctly. This approach exposed integration issues earlier than a screen-by-screen demonstration would have done. It also led to stronger emphasis on role permissions, stale sensor handling, explicit incident states and failure-safe UI states.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design integrated architecture	Layered architecture connecting sensors, API, intelligence, Digital Twin and response	Fully Achieved
Collect environmental readings	ESP32 prototype with rain, water, flame, gas and temperature/humidity sensors	Fully Achieved
Risk detection	Configurable threshold/risk evaluation and anomaly-oriented workflow	Fully Achieved at prototype level
Digital Twin	Zone-based GIS visualization and live state model	Fully Achieved at prototype level
Responder allocation	Zone/availability/skill/workload recommendation workflow	Fully Achieved at prototype level

Route planning	A* route calculation with blocked-road handling	Fully Achieved at prototype level
Citizen reporting	Photo report, token and status workflow	Fully Achieved at prototype level
Security	Role-based restrictions and protected API access	Fully Achieved for tested prototype roles
Validated ML accuracy	Production-grade model accuracy on a representative city dataset	Not claimed / Future work

4.7 Strengths & Limitations

Strengths:

- End-to-end physical-to-digital integration rather than a dashboard-only prototype.
- Modular architecture separating sensing, storage, intelligence, geospatial and response responsibilities.
- Configurable risk logic that is easier to test and explain than an unsupported black-box model.
- Digital Twin and route components provide spatial context for operational decisions.
- Role-aware workflows protect internal operational information from citizen users.
- Incident, responder and citizen workflows connect detection to action.

Limitations:

- Prototype sensor coverage is limited and does not represent city-wide sensing.
- Advanced predictive models require larger, validated datasets than those available for the prototype.
- Prototype zone labels are demonstration constructs and are not official municipal risk classifications.
- External service availability and network conditions can affect real-time behaviour.
- High-impact emergency decisions still require authorized human verification.
- Field deployment would require stronger reliability, security, governance, monitoring and disaster-recovery controls.

4.8 Project Planning, Roles & Budget

Phase	Major Activity	My Contribution
1	Problem analysis and architecture	Defined system boundary, integration interfaces and core workflow.

2	Backend/data foundation	Coordinated sensor/API/data flow and integration requirements.
3	Intelligence and Digital Twin	Connected risk state, geospatial representation and response decisions.
4	Response workflows	Integrated incident, responder, route and notification transitions.
5	Security and quality	Reviewed RBAC, privacy, protected operations and failure states.
6	Testing and deployment	Performed end-to-end validation, debugging and deployment/build verification.

Budget Item	Estimated Cost (₹)	Actual / Project Basis
ESP32 DevKit	500	Prototype purchase basis
Breadboard	180	Prototype purchase basis
Rain sensors	120	Prototype purchase basis
Water-level sensor	130	Prototype purchase basis
Flame sensor	100	Prototype purchase basis
MQ-2 gas sensor	180	Prototype purchase basis
DHT sensor	100	Prototype purchase basis
Buzzer + RGB LED + resistors	70	Prototype purchase basis
Jumper wires	300	Prototype purchase basis
USB/power accessories	100	Prototype purchase basis
Approximate hardware total	1,780	As recorded in project hardware planning

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

SentinelAI demonstrates how an urban disaster-management prototype can move beyond isolated sensing or visualization by connecting physical observations to intelligent and operational workflows. The implemented system combines an ESP32 sensing layer, REST-based data ingestion, persistent storage, configurable risk evaluation, Digital Twin visualization, incident management, responder recommendation, route planning and citizen reporting.

The core engineering contribution documented in this individual report is the integration of these modules into a coherent end-to-end system. The architecture was designed so that a physical sensor event can propagate through the backend and intelligence layers and become an operationally meaningful state. This system-level view was essential for testing because it exposed interface failures and security issues that would not be visible in isolated module demonstrations.

The prototype achieved its principal capstone objective: demonstrating a practical predictive-disaster-intelligence workflow at prototype scale. It does not claim certified emergency-service readiness or city-scale predictive accuracy. Instead, it provides a technically defensible foundation that can be extended with validated datasets, richer models and real institutional integrations.

5.2 Future Work

- Integrate validated rainfall, flood, weather and geospatial datasets for measurable predictive-model evaluation.
- Introduce spatiotemporal machine-learning models and compare them with the current transparent risk baseline.
- Add authorized satellite-derived flood and land-use information.
- Integrate authorized CCTV/computer-vision feeds and drone situational awareness where legally and operationally permitted.
- Develop a higher-fidelity 3D Digital Twin for urban assets and infrastructure.
- Extend multilingual public alerts and voice-based citizen reporting.
- Improve offline buffering, fault tolerance and deployment observability.

- Add stronger auditability, signed evidence records, MFA, key management and formal security testing.
- Integrate with authorized emergency-service systems only after governance, privacy and safety validation.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned to design a system around interfaces and state transitions rather than isolated features.
- Developed practical understanding of IoT-to-web data pipelines and spatial data integration.
- Improved knowledge of Digital Twin concepts, multi-agent coordination and disaster-response workflows.
- Strengthened understanding of role-based security, privacy-aware system design and testing discipline.
- Learned to distinguish prototype evidence from claims that require field-scale datasets and validation.

Engineering & Professional Skills Developed

- System architecture and modular decomposition.
- Full-stack integration and API contract reasoning.
- Debugging and end-to-end verification.
- Geospatial reasoning and algorithmic route planning.
- Technical documentation and evidence-based reporting.
- Engineering trade-off analysis, risk assessment and professional communication.

Individual Reflection

The most difficult engineering problem was maintaining a reliable chain from the physical sensor event to the final response state. A local component could appear correct while the overall workflow still failed because of schema, permission, state or integration mismatches. I addressed this by testing the system progressively from ingestion through storage, risk evaluation, Digital Twin update and response coordination rather than validating only the final interface.

A key engineering decision I contributed to was keeping the risk layer configurable and transparent at prototype scale instead of presenting an unvalidated black-box model as production

AI. This decision was supported by the limited training-data situation and by the need to make threshold-triggered behaviour reproducible during demonstration.

I independently strengthened my understanding of Digital Twins, REST API integration, PostgreSQL/PostGIS, role-based access control, live application state and emergency-response workflows. The project also taught me that engineering quality depends on security, failure handling and documentation as much as on feature count.

If the project were repeated, I would establish the data and evaluation strategy earlier. A validated historical dataset, explicit performance metrics and automated integration tests would make it possible to quantify prediction quality and response improvements instead of relying mainly on functional prototype validation.

REFERENCES

1. M. Batty, “Digital twins,” Environment and Planning B: Urban Analytics and City Science, vol. 45, no. 5, pp. 817–820, 2018, doi: 10.1177/2399808318796416.
2. C. Fan, C. Zhang, A. Yahja, and A. Mostafavi, “Disaster city digital twin: A vision for integrating artificial and human intelligence for disaster management,” International Journal of Information Management, vol. 56, p. 102049, 2021, doi: 10.1016/j.ijinfomgt.2019.102049.
3. D. Li, W. Yu, and Z. Shao, “Smart city based on digital twins,” Computational Urban Science, vol. 1, no. 1, p. 4, 2021, doi: 10.1007/s43762-021-00005-y.
4. L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A survey,” Computer Networks, vol. 54, no. 15, pp. 2787–2805, 2010, doi: 10.1016/j.comnet.2010.05.010.
5. A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, “Internet of Things: A survey on enabling technologies, protocols, and applications,” IEEE Communications Surveys & Tutorials, vol. 17, no. 4, pp. 2347–2376, 2015, doi: 10.1109/COMST.2015.2444095.
6. S. D. Ramchurn et al., “A disaster response system based on human-agent collectives,” Journal of Artificial Intelligence Research, vol. 57, pp. 661–708, 2016, doi: 10.1613/jair.5098.
7. N. Kankanamge, T. Yigitcanlar, A. Goonetilleke, and M. Kamruzzaman, “Determining disaster severity through social media analysis: Testing the methodology with South East Queensland flood tweets,” International Journal of Disaster Risk Reduction, vol. 42, p. 101360, 2020, doi: 10.1016/j.ijdrr.2019.101360.
8. L. Natsvlishvili, N. Jorjiashvili, and V. Kochoradze, “Development of a PostGIS-based method for creating risk maps of natural disasters using the example of Georgia,” Geodesy and Cartography, vol. 48, no. 2, pp. 70–77, 2022, doi: 10.3846/gac.2022.14791.
9. X. Yan, K. Xu, W. Feng, et al., “A rapid prediction model of urban flood inundation in a high-risk area coupling machine learning and numerical simulation approaches,” International Journal of Disaster Risk Science, vol. 12, pp. 903–918, 2021, doi: 10.1007/s13753-021-00384-0.
10. J. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed. Chichester, U.K.: Wiley, 2009.
11. ISO 22320, Security and resilience — Emergency management — Requirements for incident response, International Organization for Standardization.
12. ISO/IEC 27001, Information security, cybersecurity and privacy protection — Information security management systems — Requirements, International Organization for Standardization.
13. IEEE Std 29148, Systems and software engineering — Life cycle processes — Requirements engineering, IEEE.

APPENDICES

Appendix A – Detailed Implementation Evidence

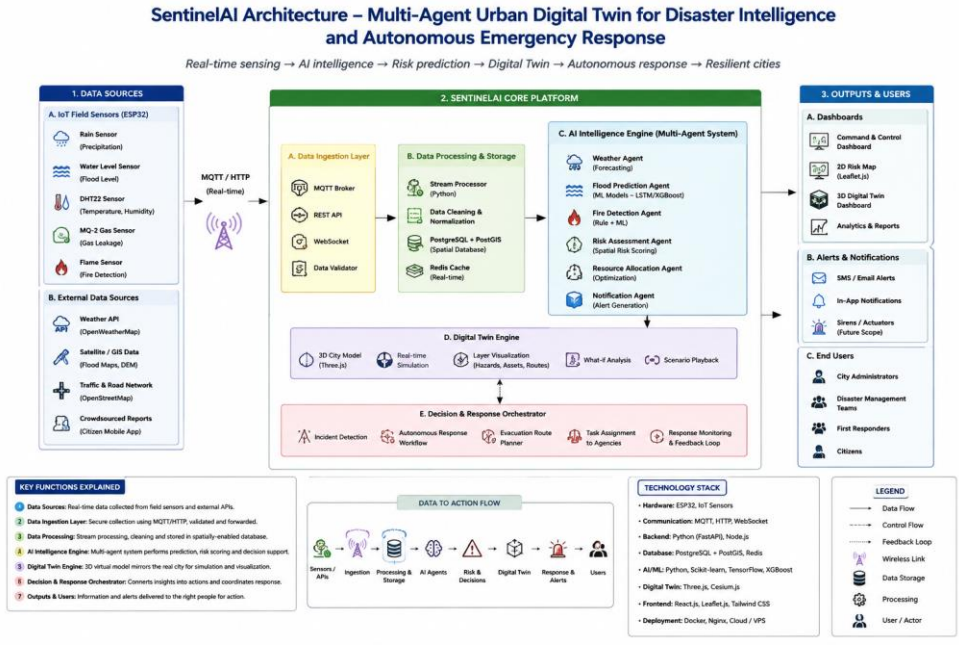

Figure A.1 System architecture evidence from the implemented SentinelAI project.

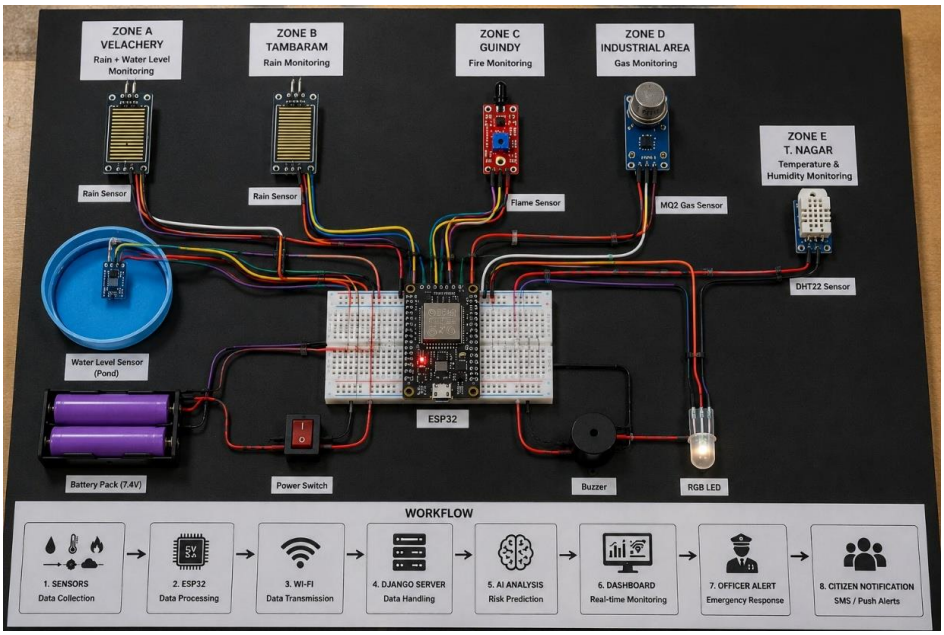

Figure A.2 Physical prototype evidence from the implemented SentinelAI project.

The architecture evidence shows the major data acquisition, data processing, AI disaster detection, intelligent decision, monitoring/analysis and emergency-response stages.

Appendix B – Core API / Interface Summary

Endpoint / Interface	Purpose
POST /api/v1/sensor-data/	Receive IoT sensor data.
GET /api/v1/zones/	Retrieve monitored zones.
GET /api/v1/sensors/	Retrieve sensor/device status.
GET /api/v1/predictions/	Retrieve risk predictions.
POST /api/v1/incidents/	Create an incident.
PATCH /api/v1/incidents/{id}/	Update incident state.
GET /api/v1/officers/	Retrieve officer availability.
POST /api/v1/assignments/	Assign/recommend responder.
POST /api/v1/citizen-reports/	Create citizen report.
GET /api/v1/reports/{token}/	Track public report status.
POST /api/v1/routes/	Calculate response/evacuation route.
GET /api/v1/analytics/	Retrieve operational analytics.

Appendix C – Incident and Response Logic

The operational chain is: detect → validate → assess risk → create/prioritize incident → identify eligible responder → assign/recommend → start response timer → accept/in-progress → route → resolve → close → record audit/performance information.

The responder recommendation considers the incident zone, officer availability, skill compatibility, distance or last known operational location, workload and configured performance factors. The project working record gives an example suitability model of 40% availability, 25% distance, 20% skill match, 10% workload and 5% historical performance; these weights are prototype configuration rather than an official emergency-service policy.

Appendix D – Hardware Components

Component	Purpose / Role
ESP32 DevKit V1	Central controller and Wi-Fi data transmission
Rain Sensor	Rainfall condition detection
Water Level Sensor	Flood/water accumulation demonstration
Flame Sensor	Fire condition demonstration
MQ-2 Gas Sensor	Gas-leak condition demonstration
DHT11/DHT22-class Sensor	Temperature and humidity observation
Buzzer	Local alert indication
RGB LED	Local state indication
Breadboard + jumper wires	Prototype interconnection
Battery pack / USB power	Prototype power source

Appendix E – Experimental / Raw Data Record Structure

The prototype stores sensor observations with device/zone context, value and timestamp. For a production evaluation, each experiment should additionally record the configured threshold, observed input, risk state, incident state, assignment timestamp, acceptance timestamp, route calculation time and final resolution time. This structure supports later quantitative evaluation without fabricating values in the capstone report.

Appendix F – Individual Contribution Evidence

Evidence Item	What It Demonstrates	Individual Relevance
System architecture figure	Layered integration of sensing, data, intelligence, geospatial and response modules.	Supports architecture and integration contribution.
Physical prototype photograph	Actual ESP32 and sensor integration.	Supports physical-to-digital integration and demonstration work.
API/interface design	Defined boundaries between sensing, backend and response services.	Supports system integration responsibility.
End-to-end test matrix	Verification across sensor, risk, incident, responder, route, citizen and security workflows.	Supports testing and engineering judgment.
Deployment/build verification records	Evidence that the software was packaged and validated for demonstration.	Supports integration and deployment responsibility.
Individual reflection	Personal engineering decisions, learning and limitations.	Supports independent learning outcome.

Appendix G – Capstone Project Outcome Mapping

Outcome / Area	Evidence in This Individual Report
Complex engineering problem formulation	Chapter 1 problem statement and constraints
Engineering design under constraints	Chapter 3 requirements, alternatives, trade-offs and risk register
Written/oral technical communication	Whole report, tables, figures and viva preparation
Ethics and professional responsibility	Chapter 3.7 and declaration
Teamwork and leadership	Chapter 4.8 roles and integration responsibilities
Experimentation and engineering judgment	Chapter 4.3–4.5 test plan and analysis
Independent acquisition of knowledge	Chapter 5.3 reflection
Standards	Chapter 3.2
Sustainability and societal impact	Chapter 3.7
Risk assessment and mitigation	Chapter 3.7
Security considerations	Chapter 3.7 and Appendix F
Integrated/system-level engineering	Chapter 3.5 and Figures 3.1/A.1
Project planning and management	Chapter 4.8
Individual contribution	Contribution Statement, Chapter 4 and Appendix F